Persistent AI Memory with LangGraph, Supabase and Ollama

Complete VS Code project guide: Python, LangGraph PostgreSQL checkpointer, Supabase, and a local Ollama model.

1. Architecture

You can integrate LangGraph, Supabase PostgreSQL and Ollama to build an AI agent with persistent memory. The conversation history remains available even after you restart your Python application.

The workflow:

```
User
  |
  v
Python Application
  |
  v
LangGraph Agent
  |
  v
Ollama LLM
  |
  v
Generate Answer
  |
  v
LangGraph Checkpointer
  |
  v
Supabase PostgreSQL
 (Persistent Memory)
```

What happens:

- The user asks a question.
- LangGraph loads the previous conversation for that conversation's ID.
- Ollama generates an answer using the conversation history.
- LangGraph saves the updated conversation state in Supabase.
- When you restart the application, the conversation can be resumed using the same conversation ID.

2. VS Code project structure

```
langgraph-supabase-memory/
|
|-- .env
|-- requirements.txt
|-- main.py
|-- README.md
```

3. Install dependencies

Create requirements.txt:

```
langgraph
langgraph-checkpoint-postgres
langchain-ollama
psycopg[binary,pool]
python-dotenv
```

Install the packages in your VS Code terminal:

```
pip install -r requirements.txt
```
Make sure Ollama is installed and running. Download a model if you haven't already:

```
ollama pull llama3.2
```
Start the Ollama server if it isn't already running:

```
ollama serve
```
Ollama will typically run at http://localhost:11434.

4. Configure Supabase

Open your Supabase project and locate its PostgreSQL connection details. You'll need a database connection string. Prefer the direct connection or an appropriate session pooler connection string supported by your network.

Create a .env file:

```
SUPABASE_DB_URI=postgresql://postgres:YOUR_PASSWORD@YOUR_DB_HOST:5432/postgres?sslmode=require

OLLAMA_MODEL=llama3.2
```
Replace the placeholders with your actual Supabase database credentials.

Important: Keep your .env file private. Never commit your database password to GitHub.

5. Complete Python code: main.py

This example uses LangGraph's PostgreSQL checkpointer to persist the conversation state.

```
import os
import uuid

from dotenv import load_dotenv
from typing import Annotated, TypedDict

from langchain_core.messages import (
    AnyMessage,
    HumanMessage,
    SystemMessage,
)
from langchain_ollama import ChatOllama

from langgraph.graph import StateGraph, START, END
from langgraph.graph.message import add_messages

from langgraph.checkpoint.postgres import PostgresSaver


# ----------------------------------------
# 1. Load environment variables
# ----------------------------------------

load_dotenv()

DB_URI = os.environ["SUPABASE_DB_URI"]
MODEL_NAME = os.getenv("OLLAMA_MODEL", "llama3.2")


# ----------------------------------------
# 2. Define LangGraph state
# ----------------------------------------

class AgentState(TypedDict):
    messages: Annotated[list[AnyMessage], add_messages]


# ----------------------------------------
# 3. Initialize Ollama LLM
```

```
# ----------------------------------------

llm = ChatOllama(
    model=MODEL_NAME,
    temperature=0
)


# ----------------------------------------
# 4. Define agent node
# ----------------------------------------

def chatbot(state: AgentState):

    system_prompt = SystemMessage(
        content=(
            "You are a helpful AI assistant. "
            "Use the supplied conversation history "
            "to answer the user's latest question."
        )
    )

    response = llm.invoke(
        [system_prompt] + state["messages"]
    )

    return {
        "messages": [response]
    }


# ----------------------------------------
# 5. Build LangGraph
# ----------------------------------------

builder = StateGraph(AgentState)

builder.add_node("chatbot", chatbot)

builder.add_edge(START, "chatbot")
builder.add_edge("chatbot", END)


# ----------------------------------------
# 6. Connect LangGraph to Supabase
# ----------------------------------------

def main():

    with PostgresSaver.from_conn_string(DB_URI) as checkpointer:

        # Create the checkpoint tables if necessary.
        checkpointer.setup()

        # Compile the graph with persistent storage.
        graph = builder.compile(
            checkpointer=checkpointer
        )

        print("\nLangGraph + Supabase Memory")
        print("Type 'exit' to stop.")
        print("Use the same thread ID to resume a conversation.\n")

        # A stable thread ID identifies a conversation.
        thread_id = input(
            "Enter conversation ID (or press Enter for a new one): "
        ).strip()

        if not thread_id:
            thread_id = str(uuid.uuid4())
```

```
        config = {
            "configurable": {
                "thread_id": thread_id
            }
        }

        print(f"\nConversation ID: {thread_id}\n")

        while True:

            user_input = input("You: ").strip()

            if user_input.lower() in ["exit", "quit"]:
                break

            if not user_input:
                continue

            # Invoke the graph with the latest user message.
            result = graph.invoke(
                {
                    "messages": [
                        HumanMessage(content=user_input)
                    ]
                },
                config=config
            )

            # Print the latest assistant response.
            print(
                "\nAssistant:",
                result["messages"][-1].content,
                "\n"
            )


if __name__ == "__main__":
    main()
```

How the persistence works

The key part is the checkpointer, which configures PostgreSQL-backed checkpoints:

```
with PostgresSaver.from_conn_string(DB_URI) as checkpointer:

    checkpointer.setup()

    graph = builder.compile(
        checkpointer=checkpointer
    )
```

The next important part is the thread ID, which identifies the conversation whose state LangGraph should retrieve and update:

```
config = {
    "configurable": {
        "thread_id": thread_id
    }
}
```

Do not generate a new thread ID for every message. Reuse the same ID when continuing the same conversation.

6. Test the persistent memory

Run the application with python main.py. Enter a new conversation ID or press Enter to generate one.

First interaction

```
You: My name is Rahul.
```

```
A: Nice to meet you, Rahul!
```

Second interaction

```
You: What is my name?

A: Your name is Rahul.
```

LangGraph supplies the conversation history to Ollama, allowing the model to answer the second question.

Restart Python

Stop the application and run it again with python main.py. Enter the same conversation ID:

```
Enter conversation ID (or press Enter for a new one):
```

Then ask:

```
You: What is my name?

A: Your name is Rahul.
```

This works because the checkpointed message history is stored in Supabase PostgreSQL rather than only in Python's memory. The exact response depends on the model, but the previous messages should be available to it.

7. What gets stored in Supabase?

The PostgreSQL checkpointer creates and manages checkpoint-related tables. Conceptually, the saved state looks like this (these are conceptual fields; the actual schema is managed by the LangGraph checkpointer):

Field	Example
Thread ID	conversation-001
Message 1	My name is Rahul
Message 2	Nice to meet you, Rahul
Message 3	What is my name?
Message 4	Your name is Rahul
Checkpoint state	Current LangGraph state
Metadata	Checkpoint and execution information

You can inspect the resulting tables using the Supabase SQL Editor.

8. Persistent memory vs. long-term AI memory

Feature	PostgreSQL checkpointer	Long-term memory store
Save conversation state	Yes	Not its primary purpose
Resume a conversation	Yes	Not by itself
Preserve history after restart	Yes	Depends on implementation
Share selected facts across different conversations	Not automatically	Yes, when implemented
Store user preferences	Possible in conversation history	Yes, as explicit user memories
Automatically retrieve facts from every conversation	No	Requires retrieval logic

The code above implements persistent conversational memory, not a separate cross-conversation knowledge store. For example, suppose the user has two conversations:

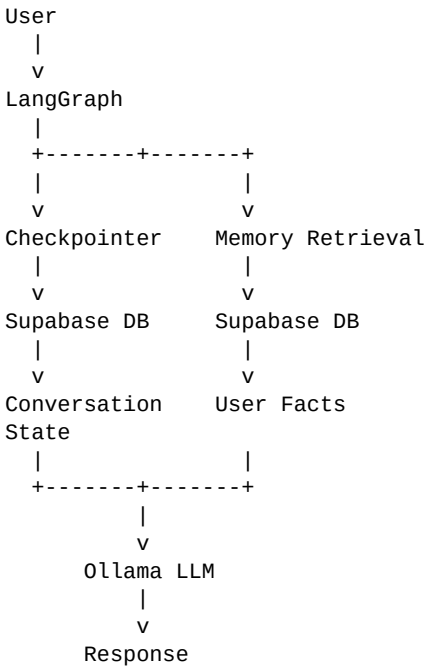
```
Thread A:
```

```
My name is Rahul.
I work with Azure AI.

Thread B:
What cloud platform do I use?
```

If Thread B has a different thread_id, LangGraph will not automatically load Thread A's history. To support that use case, add a long-term memory store in Supabase, keyed by user ID, and retrieve relevant facts in your agent before calling Ollama.

Recommended architecture



Recommendation: start with the working checkpointer implementation above. Then add a separate long-term memory store if you need user facts and preferences to be shared across multiple conversations.